HAFTA 1

Algoritmik Düşünme ve Veri Yapılarına Giriş

Kod yazmadan önce bir problem üzerine nasıl düşüneceğiniz — ve veriyi organize etme biçiminizin sonrasındaki her şeyi nasıl değiştirdiği.

Bugün Neler Göreceğiz

1. **Bir algoritma nedir, "iyi" bir algoritmayı ne yapar**

   Her doğru algoritmanın paylaştığı özellikler

2. **Algoritmik problem çözme süreci**

   Belirsiz bir iş talebinden çalışan koda

3. **Sözde kod (pseudocode) ve akış şemaları**

   Python yazmadan önce bir çözüm tasarlamak

4. **Bir veri yapısı nedir, seçim neden önemlidir**

   Gerçekten ihtiyaç duyduğunuz işlemler için veriyi organize etmek

5. **Soyut Veri Tipleri (ADT)**

   Bir yapının ne yaptığını nasıl yapıldığından ayırmak

6. **Hesaplamalı düşünmenin dört ayağı**

   Ayrıştırma, örüntü tanıma, soyutlama, algoritmalar

01 / 05

Algoritma Nedir?

TEMEL KAVRAM

Algoritma

Algoritma

Verilen bir girdiyi istenen bir çıktıya dönüştüren, sonlu, sıralı ve iyi tanımlanmış adımlar dizisidir; ayrıca sonlanacağı garanti edilir.

Örnek

"Ödenmemiş faturaların bir listesi verildiğinde, bir müşterinin şu anda borçlu olduğu toplam tutarı üretin."
Bu bir problem tanımıdır — algoritma, bunu çözen kesin adımlar dizisidir.

TEMEL KAVRAM

İyi Bir Algoritmanın Beş Özelliği

- **Girdi — sıfır veya daha fazla iyi tanımlanmış girdi alır**
- Çıktı — en az bir iyi tanımlanmış çıktı üretir
- Kesinlik — her adım kesin ve açıktır
- Sonluluk — sonlu sayıda adımdan sonra sona erer
- Etkililik — her adım, elle veya makineyle tam olarak gerçekleştirilebilecek kadar temeldir

Neden Önemli

Bu beş özellikten biri eksikse, elinizde bir bilgisayarın (veya yeni bir çalışanın) güvenilir biçimde yürütebileceği bir algoritma değil, belirsiz bir plan vardır.

İŞ SENARYOSU

Kredi Onayı İçin Bir Algoritma Tasarlamak

Bir bankanın bilgi sistemi, bir bireysel kredi başvurusunun tutarlı ve adil biçimde ön onay alıp almayacağına karar vermeli.

- Girdi: başvuranın geliri, mevcut borcu, kredi skoru
- Adımlar: borç/gelir oranını hesapla, kredi skorunu bir eşikle karşılaştır, iki kontrolü birleştir
- Çıktı: "onaylandı", "reddedildi" veya "manuel incelemeye gerek var"
- Her başvuran tam olarak aynı, iyi tanımlanmış adımlardan geçmelidir — algoritmanın değeri bu tutarlılıktır

İŞ AKIŞI

Algoritmik Problem Çözme Süreci

- **1. Problemi anlayın — girdi nedir, istenen çıktı nedir?**
- 2. Kod yazmadan önce düz dilde veya sözde kodla bir çözüm planlayın
- 3. İlgili veri için uygun veri yapılarını seçin
- 4. Planı koda çevirin
- 5. Normal durumlar, uç durumlar ve geçersiz girdilerle test edin
- 6. Önce doğruluk, sonra verimlilik için iyileştirin

02 / 05

Sözde Kod ve Akış Şemaları

TEMEL KAVRAM

Sözde Kod

Pseudocode (Sözde Kod)

Bir algoritmanın mantığını, belirli bir dilin sözdizimini düşünmeden, yapılandırılmış düz dil ifadeleriyle betimleyen, dilden bağımsız gayriresmî bir yoldur — koda dökülebilecek kadar kesindir.

Örnek

```
BAŞLA
  sipariş tutarları listesini OKU
  toplam DEĞERİNİ 0 YAP
  listedeki HER tutar İÇİN
     toplam = toplam + tutar
  DÖNGÜYÜ BİTİR
  toplamı YAZDIR
BİTİR
```

UYGULAMA

Sözde Koddan Python'a

```
# Sözde kod:
#   toplam DEĞERİNİ 0 YAP
#   siparis_listesindeki HER tutar İÇİN
#       toplam = toplam + tutar
#   toplamı YAZDIR

siparis_listesi = [120.50, 980.00, 45.25, 300.75]
toplam = 0
for tutar in siparis_listesi:
    toplam = toplam + tutar
print(toplam)
```

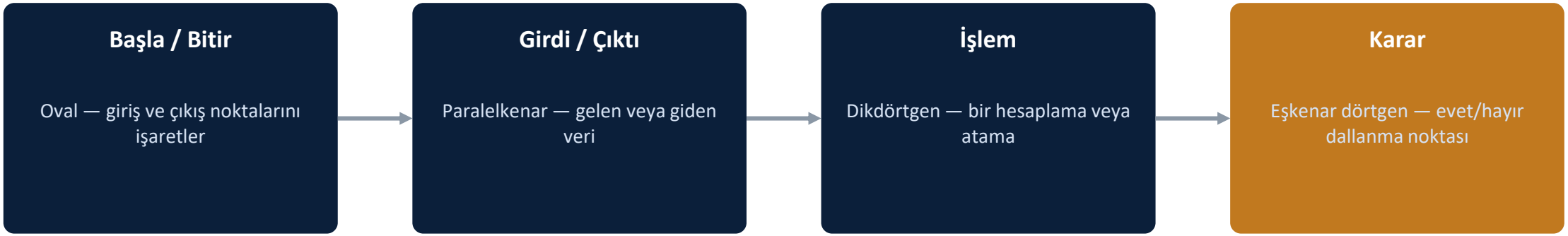
Çıktı

```
1446.5
```

Python kodunun sözde koda ne kadar yakın olduğuna dikkat edin — sözde kodu önce yazmanın amacı tam olarak bu yakınlıktır.

DIYAGRAM

Bu Dönem Kullanacağınız Akış Şeması Sembolleri



İŞ SENARYOSU

Bir Kargo Ücreti Kararı Tasarlamak

Bir çevrim içi mağaza kargo ücretini hesaplamalı: bir harcama eşiğinin üzerinde ücretsiz, aksi halde sabit bir ücret, büyük boy ürünlerde ek bir ücret farkıyla birlikte.

- Önce akış şeması olarak taslak çıkarın: Başla → Sipariş tutarı ve boyutu girdisi → Karar (eşik üstü mü?) → Karar (büyük boy mu?) → Çıktı ücret → Bitir
- Akış şeması üzerinde anlaştıktan sonra sözde kodu, ardından Python'ı yazın
- Bu sıra — koddan önce akış şeması/sözde kod — mantık hatalarını, düzeltmesi hâlâ ucuzken yakalar

Uygulama: Önce Sözde Kod

Not defterinizi açmadan önce (kağıda veya bir metin hücresine) şunun için sözde kod yazın:

- Bir çalışan maaşları listesini okumak
- Ortalama maaşı hesaplamak
- Ortalamadan %50'den fazla yüksek olan herhangi bir maaşı "aykırı değer" olarak işaretlemek
- Ortalamayı ve aykırı değerler listesini yazdırmak

İpucu

Şu şekilde yapılandırın: DEĞER YAP, HER... İÇİN, EĞER, YAZDIR — fatura toplamı örneğiyle aynı kalıplar.

03 / 05

Veri Yapısı Nedir?

TEMEL KAVRAM

Veri Yapısı

Veri Yapısı

Bir programın veri üzerinde gerçekleştirmesi gereken işlemler için verimli bir şekilde kullanılabilmesi amacıyla, veriyi bellekte organize etme, saklama ve erişme biçimidir.

Örnek

Bir telefon rehberi kişileri alfabetik olarak organize eder çünkü en sık yaptığınız işlem — birini isme göre bulmak — bu şekilde hızlıdır. Aynı isimler katılım tarihine göre organize edilse bu arama yavaş olurdu.

TEMEL KAVRAM

Veri Yapısı Seçimi Neden Önemlidir

- Aynı veri, farklı organize edildiğinde bir işlemi anlık ya da acı verici derecede yavaş hale getirebilir
- Her yapı bir ödünleşim sunar: hızlı arama mı, düşük bellek kullanımı mı, hızlı ekleme mi — nadiren üçü birden
- Kötü seçim yalnızca performans kaybettirmez — büyük bir sistemde bazı özellikleri neredeyse imkânsız hale getirebilir
- Bu ders, her problemi bir listeye sıkıştırmak yerine doğru aracı seçebilmeniz için size bir yapı araç kutusu öğretmek için var

KARŞILAŞTIRMA

Ham Veri ile Organize Edilmiş Bir Veri Yapısı

Organize Edilmemiş Veri

- Geliş sırasına göre bir müşteri kaydı yığını
- Bir müşteriyi bulmak her kaydı kontrol etmek demek
- Yeni kayıt eklemek kolay — ama her şey diğer yavaş

Bir Yapı İçinde Organize Edilmiş

- Müşteri ID'sine göre indekslenmiş kayıtlar (örn. bir sözlük)
- Bir müşteriyi bulmak neredeyse anlık
- Yapı, en önemli işlem için seçildi: arama

YOL HARITASI

Bu Dönem Kuracağımız Yapıların Ön İzlemesi

Hafta	Yapı	Tipik İş Kullanımı
3	Diziler / Listeler	Sıralı kayıtlar: günlük işlemler, ürün katalogları
4	Bağlı Listeler	Sık ekleme/çıkarma olan diziler, örn. görev kuyruğu
5–6	Yığınlar ve Kuyruklar	Geri alma geçmişi, yazdırma/sipariş kuyrukları, iş akışı adımları
10	Hash Tabloları	Anahtarla anlık arama: kullanıcı adları, SKU'lar, oturum jetonları
11–12	Ağaçlar ve Graflar	Organizasyon şemaları, dosya sistemleri, ağlar, öneri bağlantıları

İŞ SENARYOSU

Bir Sipariş Yönetim Sistemi İçin Yapı Seçmek

Bir YBS ekibi sipariş yönetimi arka ucunu tasarlıyor: siparişler sürekli gelir, geliş sırasına göre işlenmeli ve müşteriye göre aranabilir olmalıdır.

- Bir kuyruk, "geliş sırasına göre işle" ile doğal olarak eşleşir
- Müşteri ID'sine göre anahtarlanmış bir sözlük, "müşteriye göre ara" ile doğal olarak eşleşir
- Her iki iş için de tek bir düz liste kullanmak, birinde yavaş ve beceriksiz çözümler zorunlu kılar
- İşlemle eşleşen yapıyı seçmek, bu dersin geliştirdiği temel beceridir

04 / 05

Soyut Veri Tipleri

TEMEL KAVRAM

Soyut Veri Tipi (ADT)

Soyut Veri Tipi

Desteklediği işlemler (örn. "push", "pop") ve bu işlemlerin davranışıyla tanımlanan kavramsal bir modeldir — bu işlemlerin bellekte gerçekte nasıl uygulandığından bağımsızdır.

Örnek

Bir Yığın (Stack) ADT'si push, pop ve peek ile ve "son giren ilk çıkar" kuralıyla tanımlanır. Bunun bir Python listesi üzerine mi yoksa bağlı bir liste üzerine mi kurulduğu, bu arayüzün arkasına gizlenmiş bir uygulama ayrıntısıdır.

TEMEL KAVRAM

Arayüz ile Uygulama

ADT ("ne" sorusu)

- İzin verilen işlemleri tanımlar
- Beklenen davranış/kuralları tanımlar
- Aynı ADT'nin birden çok uygulaması olabilir

Somut Yapı ("nasıl" sorusu)

- Gerçek bellek düzeni ve kod
- Örn. Python listesi üzerine kurulu bir Yığın ile bağlı liste üzerine kurulu bir Yığın
- ADT'yi kullananların bu ayrıntıyı bilmesi gerekmemeli

TEMEL KAVRAM

ADT'ler YBS Sistemleri İçin Neden Önemlidir

- ADT olarak düşünmek, düşük seviye uygulama ayrıntılarına karar vermeden önce bir sistemin mantığını tasarlamanızı sağlar
- Bir ekibin, sistemin geri kalanını bozmadan bir uygulamayı sonradan (örn. performans için) değiştirmesine olanak tanır
- Bu dönem kuracağımız her özel veri yapısı önce bir ADT — bir işlemler kümesi — olarak, sonra somut bir uygulama olarak tanıtılacak

05 / 05

Hesaplamalı Düşünme

TEMEL KAVRAM

Hesaplamalı Düşünmenin Dört Ayağı

- Ayrıştırma — büyük bir problemi daha küçük, yönetilebilir alt problemlere bölmek
- Örüntü tanıma — bu problem ile daha önce çözdüklerinizin benzerliklerini fark etmek
- Soyutlama — önemli olana odaklanmak, ilgisiz ayrıntıyı görmezden gelmek
- Algoritma tasarımı — kesin, adım adım çözümü inşa etmek

İŞ SENARYOSU

"Bir Müşteri Siparişini İşle"yi Ayrıştırmak

Tek bir istek — "bu siparişi işle" — aslında birbirine zincirlenmiş birkaç küçük algoritmik problemdir.

- Siparişi doğrula (tüm ürünler stokta mı?)
- Toplamı hesapla (fiyatları, indirimleri, vergiyi uygula)
- Envanteri güncelle (stok sayılarını azalt)
- İşlemi kaydet (sipariş geçmişine ekle)
- Bu alt problemlerin her biri bağımsız olarak tasarlanabilir, test edilebilir ve iyileştirilebilir

UYGULAMA

Ayrıştırmayı Fonksiyonlara Dönüştürmek

```
def siparisi_dogrula(sepet, envanter):
    return all(envanter.get(urun, 0) > 0 for urun in sepet)

def toplam_hesapla(sepet, fiyatlar, vergi_orani=0.18):
    ara_toplam = sum(fiyatlar[urun] for urun in sepet)
    return ara_toplam * (1 + vergi_orani)

def siparisi_isle(sepet, envanter, fiyatlar):
    if not siparisi_dogrula(sepet, envanter):
        return "Sipariş reddedildi: ürün stokta yok"
    return round(toplam_hesapla(sepet, fiyatlar), 2)
```

Ayrıştırmadaki her alt problem kendi küçük, test edilebilir fonksiyonuna dönüşür — algoritmik düşünmenin doğrudan kod yapısına dönüşmesi budur.

TEMEL KAVRAM

Doğruluk ile Verimlilik

Önce doğruluk, sonra verimlilik

Bir algoritma, uç durumlar dahil her geçerli girdi için doğru çıktıyı üretiyorsa doğrudur. Verimlilik (ne kadar zaman/bellek kullandığı) ayrı, sonraki bir konudur — bir şeyi hızlandırmak için asla doğruluktan ödün vermeyin.

Örnek

Doğru cevabı yavaş veren bir arama yine de faydalıdır. Yanlış cevabı anında veren bir arama işe yaramaz. Verimliliği, gelecek hafta başlayarak Big-O gösterimiyle resmî olarak inceleyeceğiz.

HAFTA 2 ÖN İZLEMESI

İki Doğru Algoritma, Çok Farklı Hız

100.000 müşteriden oluşan bir listede bir müşterinin kaydını ID'ye göre bulmak için ya her kaydı tek tek tarayabilirsiniz ya da doğrudan anahtarla arayabilirsiniz.

- Doğrusal tarama: kaydı 1 kontrol et, sonra 2, sonra 3… bulana kadar — doğru, ama potansiyel olarak 100.000'in tamamını kontrol eder
- Doğrudan arama (örn. ID'ye göre anahtarlanmış bir sözlük ile): neredeyse doğrudan cevaba atlar
- İkisi de aynı problem için doğru algoritmalardır — gelecek hafta bunları kesin olarak nasıl karşılaştıracağımızı öğreneceğiz

İYI UYGULAMALAR

Algoritma Tasarımında Yaygın Tuzaklar

- Bir plan olmadan doğrudan koda geçmek — küçük mantık hataları sonradan pahalı hale gelir
- Uç durumları unutmak: boş bir liste, sıfır bir değer, eksik bir anahtar
- ADT'yi (ne yaptığını) tek bir somut uygulamayla (nasıl yapıldığıyla) karıştırmak
- Algoritma henüz doğru bile değilken hız için optimize etmek
- İşleme uygun olanı seçmek yerine alışkanlıkla bir veri yapısı seçmek ("ben zaten liste kullanırım")

REFERANS

Hızlı Bir Tasarım Kontrol Listesi

Soru	Neden Önemli
Girdi ve çıktı tam olarak nedir?	Belirsiz gereksinimler belirsiz algoritmalar üretir
Uç durumlar nelerdir?	Boş girdi, yinelenen değerler, geçersiz veri hepsi ele alınmalı
Sözde kod veya akış şeması yazdım mı?	Mantık hatalarını pahalı hale gelmeden yakalar
Bu veri üzerinde en sık hangi işlem yapılıyor?	Hangi veri yapısının gerçekten uyduğunu belirler
Hızı önemsemeden önce doğru mu?	Hızlı yanlış cevap yine de yanlıştır

BİLGİ KONTROLÜ

Hızlı Kontrol 1/2

Aşağıdakilerden hangisi iyi bir algoritmanın beş özelliğinden biri DEĞİLDİR?

A Sonluluk

B Kesinlik

C Popülerlik

D Etkililik

Why: Beş özellik girdi, çıktı, kesinlik, sonluluk ve etkililiktir — popülerlik doğrulukla hiç ilgili bir özellik değildir.

BİLGİ KONTROLÜ

Hızlı Kontrol 2/2

Bir Yığın ADT'si push/pop/peek ve "son giren ilk çıkar" ile tanımlanır. Onu Python listesi yerine bağlı liste üzerine uygulamak neyi değiştirir?

A ADT'nin tanımlı davranışını

B Sadece altta yatan uygulamayı, arayüzü değil

C push ve pop'a izin verilip verilmediğini

D Bir ADT tanımlandıktan sonra hiçbir şey değiştirilemez

Why: ADT'nin işlemleri ve kuralları sabit kalır; yalnızca somut, "nasıl inşa edildiği" uygulaması değişir.

Uygulama: Ayrıştır ve Yapı Seç

Üyelerin kitap ödünç alıp iade edebildiği bir kütüphane sistemi için, not defterinizde:

- "Bir üye kitap ödünç alır"ı 3-4 küçük alt adıma ayrıştırın
- Her alt adım için hangi veriyi okuması veya güncellemesi gerektiğini not edin
- Bir kitabı ISBN'sine göre aramaya hangi veri kabının (liste, sözlük vb.) uygun olduğunu önerin
- "Bir kitabın mevcut olup olmadığını kontrol et" adımı için sözde kod yazın

HINT

Şunu sorun: bu veri üzerinde en sık hangi işlem oluyor — arama mı, ekleme mi, çıkarma mı? Bu cevap doğru yapıya işaret eder.

Hafta 1 Özeti

- ✓ Bir algoritma, beş tanımlayıcı özelliğe sahip sonlu, kesin, sonlanan bir adımlar dizisidir
- ✓ Sözde kod ve akış şemaları, kod yazmadan önce mantığı tasarlamanızı ve kontrol etmenizi sağlar
- ✓ Bir veri yapısı, bir programın gerçekten gerçekleştirmesi gereken işlemler için veriyi organize eder
- ✓ Bir ADT, bir yapının ne yaptığını nasıl uygulandığından ayırır
- ✓ Ayrıştırma, örüntü tanıma, soyutlama ve algoritma tasarımı hesaplamalı düşünmenin dört ayağıdır

NEXT WEEK

Hafta 2: Algoritma Analizi ve Big-O Gösterimi — bir algoritmanın verimliliğini sezgisel değil, kesin olarak nasıl ölçüp karşılaştıracağımız.